YENEPOYA

(DEEMED TO BE UNIVERSITY)

Project – Low Level Design

on

Enterprise Supply Chain Dashboard

Student Name: Muhammed Fazin T.K

Industry Mentor: **Guided by:**

Table of Contents

1. Introduction. 6
   1.1. Scope of the document. 6
   1.2. Intended Audience 6
   1.3. System overview 6

2. System Design. 6
   2.1. Application Design 6
   2.2. Process Flow 6
   2.3. Information Flow. 7
   2.4. Components Design 8
   2.5. Key Design Considerations 8
   2.6. API Catalogue. 8

3. Data Design. 8
   3.1. Data Model 8
   3.2. Data Access Mechanism 9
   3.3. Data Retention Policies 9
   3.4. Data Migration 9

4. Interfaces 9

5. State and Session Management 10

6. Caching 10

7. Non-Functional Requirements 10
   7.1. Security Aspects 10
   7.2. Performance Aspects 10

8. References 11

1. INTRODUCTION

The Low-Level Design (LLD) provides a comprehensive breakdown of the internal architecture and implementation logic of the AI-Based Edge Healthcare Monitoring System. Unlike the High-Level Design, which focuses on overall system architecture, this document dives into the internal workings of each module, including function-level behavior, control flow, data handling, and interaction between components.

The system is designed using Python and integrates computer vision and deep learning technologies to perform real-time object detection. The implementation ensures modularity, scalability, and efficient resource utilization, enabling deployment on edge devices such as laptops without requiring cloud infrastructure.

1.1 Scope of the Document

This document includes:

- Detailed module-level design
- Function-level breakdown and pseudo-code
- Data structures and storage mechanisms
- Control flow and execution logic
- Error handling and system constraints
- Performance optimization strategies

It serves as a blueprint for developers implementing and maintaining the system.

1.2 Intended Audience

- **Software Developers:** For implementation and debugging
- **System Designers:** For validating architecture decisions
- **Test Engineers:** For designing test cases
- **Academic Evaluators:** For assessing technical depth

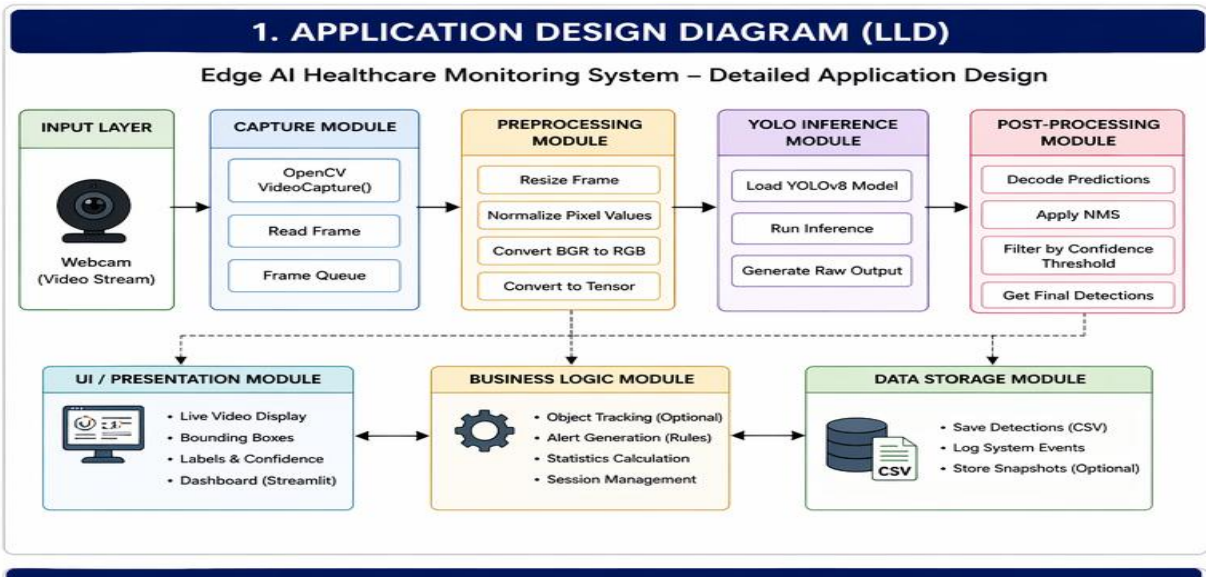
1.3 System Overview

The system operates as a real-time loop-driven application. It continuously captures frames from a webcam, processes them using a YOLO model, and displays detection results.

The workflow is controlled by a user interface that allows starting and stopping the system dynamically. The detection results are stored for analysis and visualization.

2. SYSTEM DESIGN

2.1 Application Design



The application follows a modular design pattern. Each module is responsible for a specific task and interacts with other modules through defined interfaces.

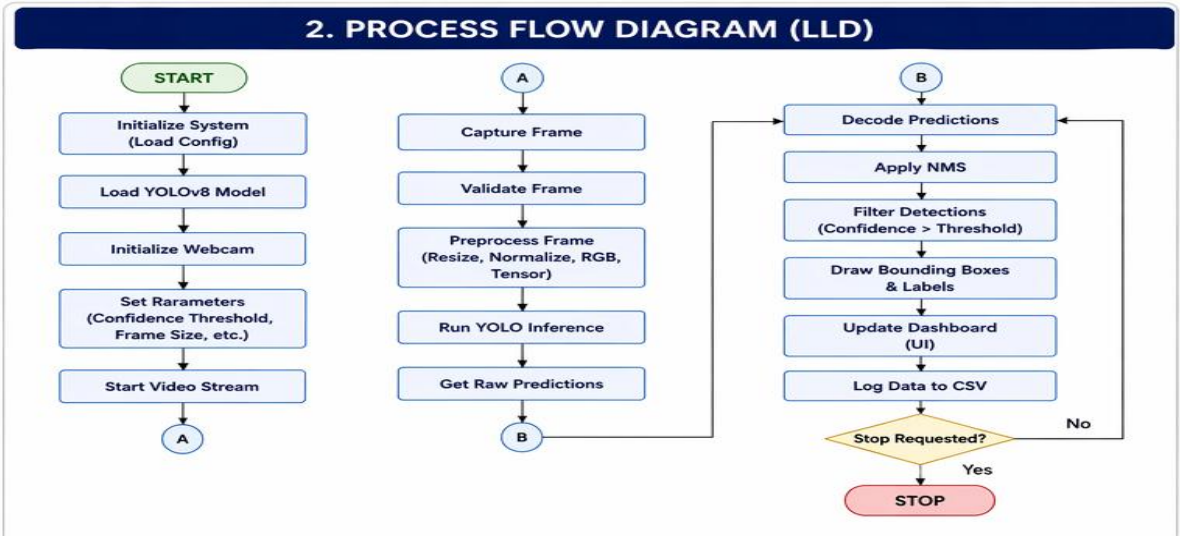
Module Breakdown

Module	Responsibility
Camera Module	Frame acquisition
Preprocessing Module	Image transformation
Detection Module	Model inference
Prediction Module	Data extraction
UI Module	Visualization
Storage Module	Data persistence

Class-Level Design (Conceptual)

```
class Camera:
    def capture_frame(self)

class Preprocessor:
    def preprocess(self, frame)

class Detector:
    def detect(self, frame)

class UI:
    def display(self, frame)

class Storage:
    def save(self, data)
```

2.2 Process Flow



The system follows a continuous loop controlled by session state.

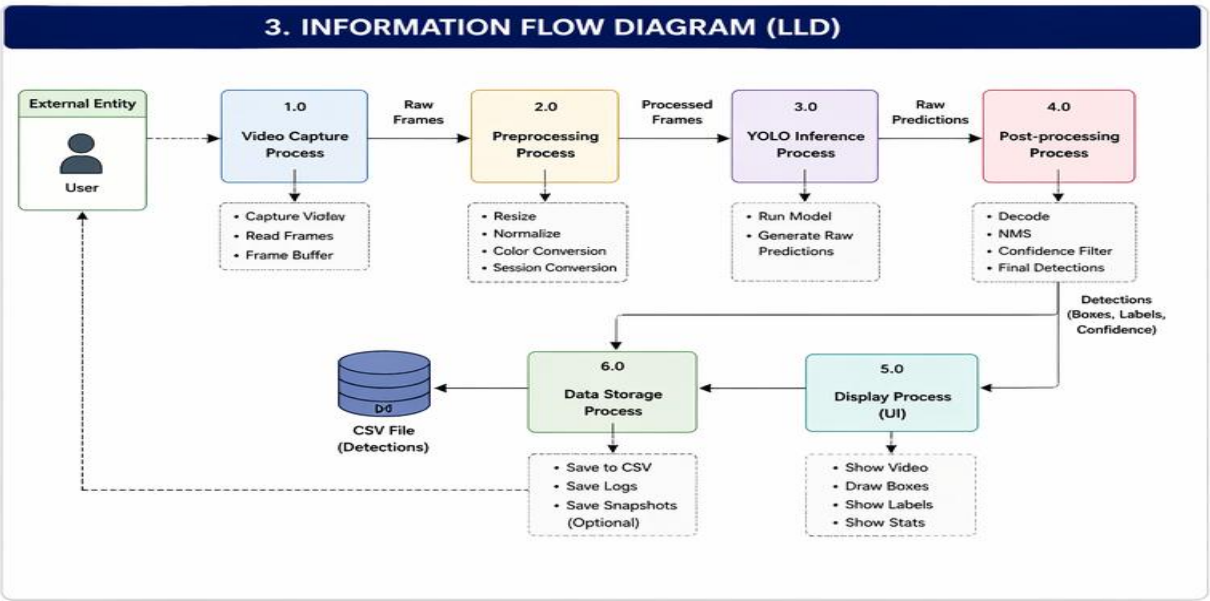
Detailed Algorithm

```
Initialize system

while system_running:
    frame = capture_frame()

    if frame is None:
        handle_error()

    processed_frame = preprocess(frame)

    detections = detect(processed_frame)

    filtered = filter(detections)

    display(filtered)

    save(filtered)
```

Loop Control Logic

- Start button → activates loop
- Stop button → breaks loop
- Session state → prevents infinite execution

2.3 Information Flow



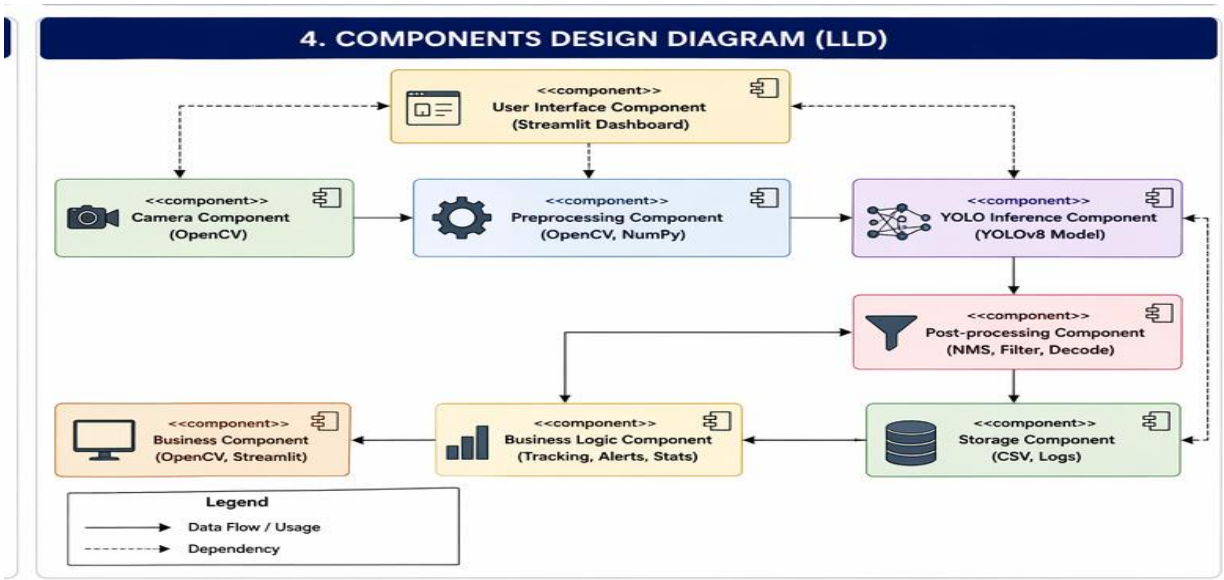
Data Transformation Pipeline

Raw Frame (NumPy Array)
↓
Preprocessed Frame
↓
Tensor Format
↓
YOLO Model Output
↓
Filtered Results
↓
Visualization + Storage

Data Types Used

Stage	Data Type
Frame	NumPy Array
Tensor	PyTorch Tensor
Output	List of Objects
Storage	DataFrame

2.4 Components Design



1. Camera Module

- Uses OpenCV
- Initializes camera using cv2.VideoCapture(0)
- Captures frames continuously

```
def capture_frame():
    ret, frame = cap.read()
    if not ret:
        return None
    return frame
```

2. Preprocessing Module

- Resizes frame to 640x640
- Normalizes pixel values
- Converts to model-compatible format

```
def preprocess(frame):
    frame = cv2.resize(frame, (640, 640))
    return frame
```

3. Detection Module

- Loads YOLO model
- Performs inference

```
def detect(frame):
    results = model(frame)
    return results
```

4. Prediction Module

- Extracts:
    - Bounding boxes
    - Labels
    - Confidence scores

```
for box in results[0].boxes:
    label = model.names[int(box.cls[0])]
    confidence = float(box.conf[0])
```

5. Display Module

- Uses Streamlit UI components
- Displays:
    - Video feed
    - Detection results
    - Charts

6. Storage Module

- Stores detection logs
- Uses Pandas

```
data.append([timestamp, label, confidence])
```

2.5 Key Design Considerations

Performance Optimization

- Lightweight model (YOLOv8n)
- Frame skipping (optional)
- Efficient memory usage

Accuracy

- Confidence threshold filtering
- Object filtering

Scalability

- Modular design allows adding:
  - Mask detection
  - Fall detection

Reliability

- Error handling for camera failure
- Safe loop termination

2.6 API Catalogue

API	Input	Output	Description
capture_frame	None	Frame	Captures image
preprocess_frame	Frame	Frame	Resizes image
detect_objects	Frame	Results	Runs YOLO
extract_data	Results	List	Extract labels
display_output	Frame	None	Shows UI
save_data	List	CSV	Stores logs

3. DATA DESIGN

3.1 Data Model

```
{
   "timestamp": "string",
   "object": "string",
   "confidence": "float"
}
```

3.2 Data Access Mechanism

- Data stored in list
- Converted to Pandas DataFrame
- Access via indexing

3.3 Data Retention Policies

- Temporary runtime storage
- Optional persistent storage
- No sensitive healthcare data

3.4 Data Migration

- Export to CSV
- Future:
    - Database (MySQL)
    - Cloud storage

4. INTERFACES

User Interface

- Streamlit-based
- Real-time dashboard

Hardware Interface

- Webcam

Software Interface

- OpenCV
- YOLO

5. STATE AND SESSION MANAGEMENT

- Uses st.session_state
- Maintains:
    - Run state
    - Detection logs

Session Logic

```
if "run" not in session:
    session["run"] = False
```

6. CACHING

Currently not implemented.

Future improvements:

- Model caching
- Frame caching

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Security Aspects

- No external communication
- Local processing
- Safe data handling

7.2 Performance Aspects

- Low latency (<100ms per frame)
- Efficient CPU usage
- Real-time processing

8. REFERENCES

- YOLO Official Docs
- OpenCV Docs
- Streamlit Docs
- Python Documentation